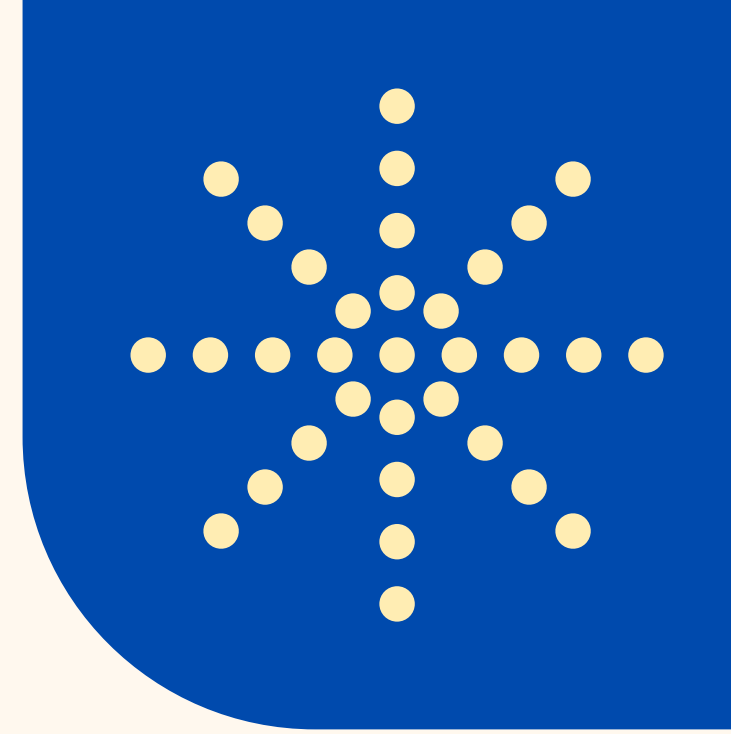
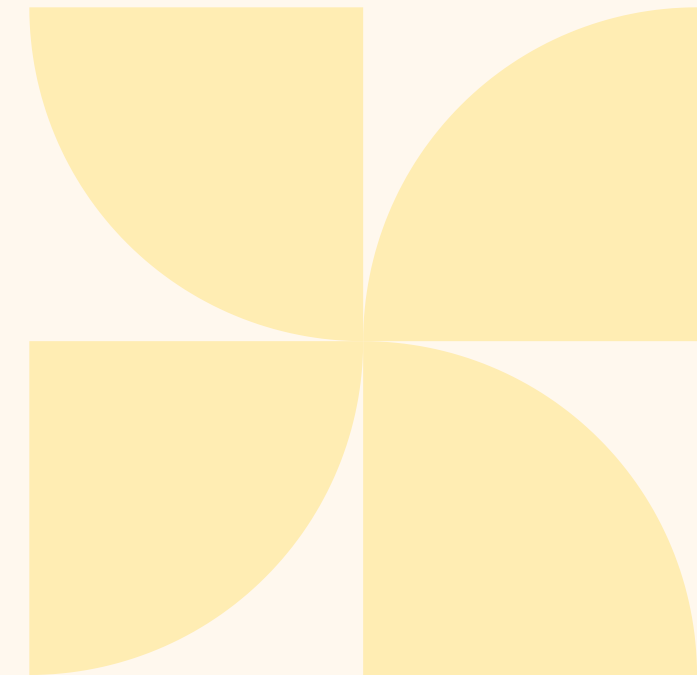
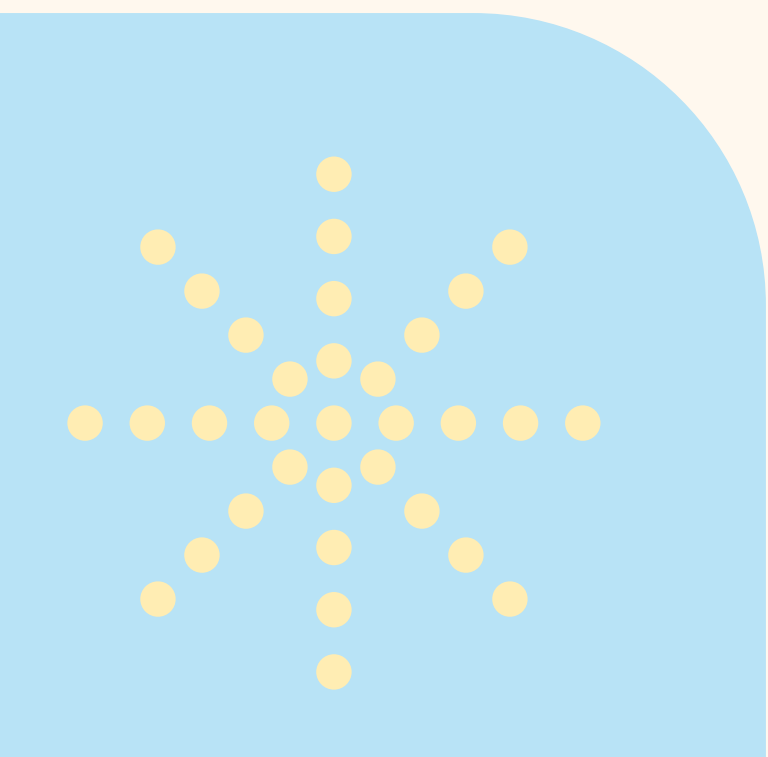


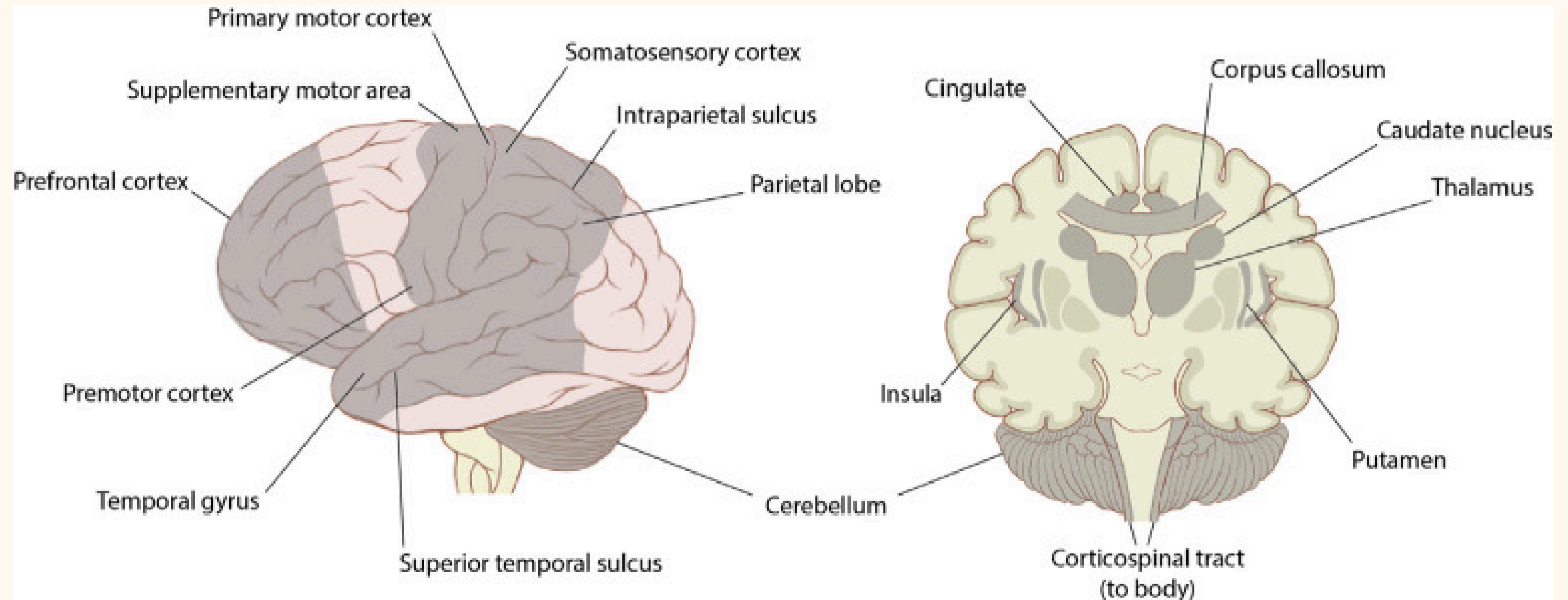


TEACHING AN RL AGENT TO DANCE

Lyons Wang, Kevin Pham, & McKenzie Green
NEUR1660 FINAL PROJECT

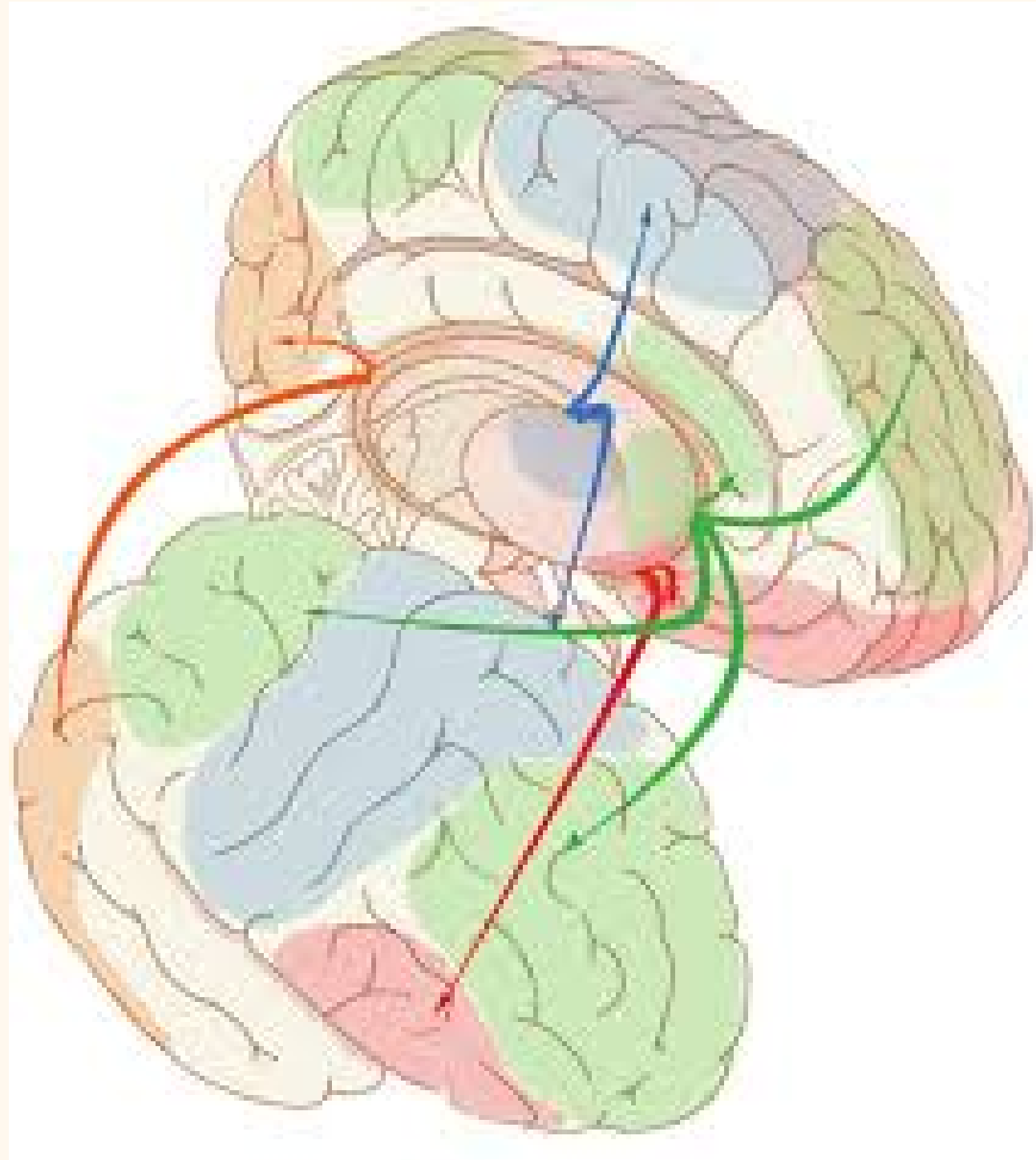


BACKGROUND



Basso et al., 2021

BACKGROUND

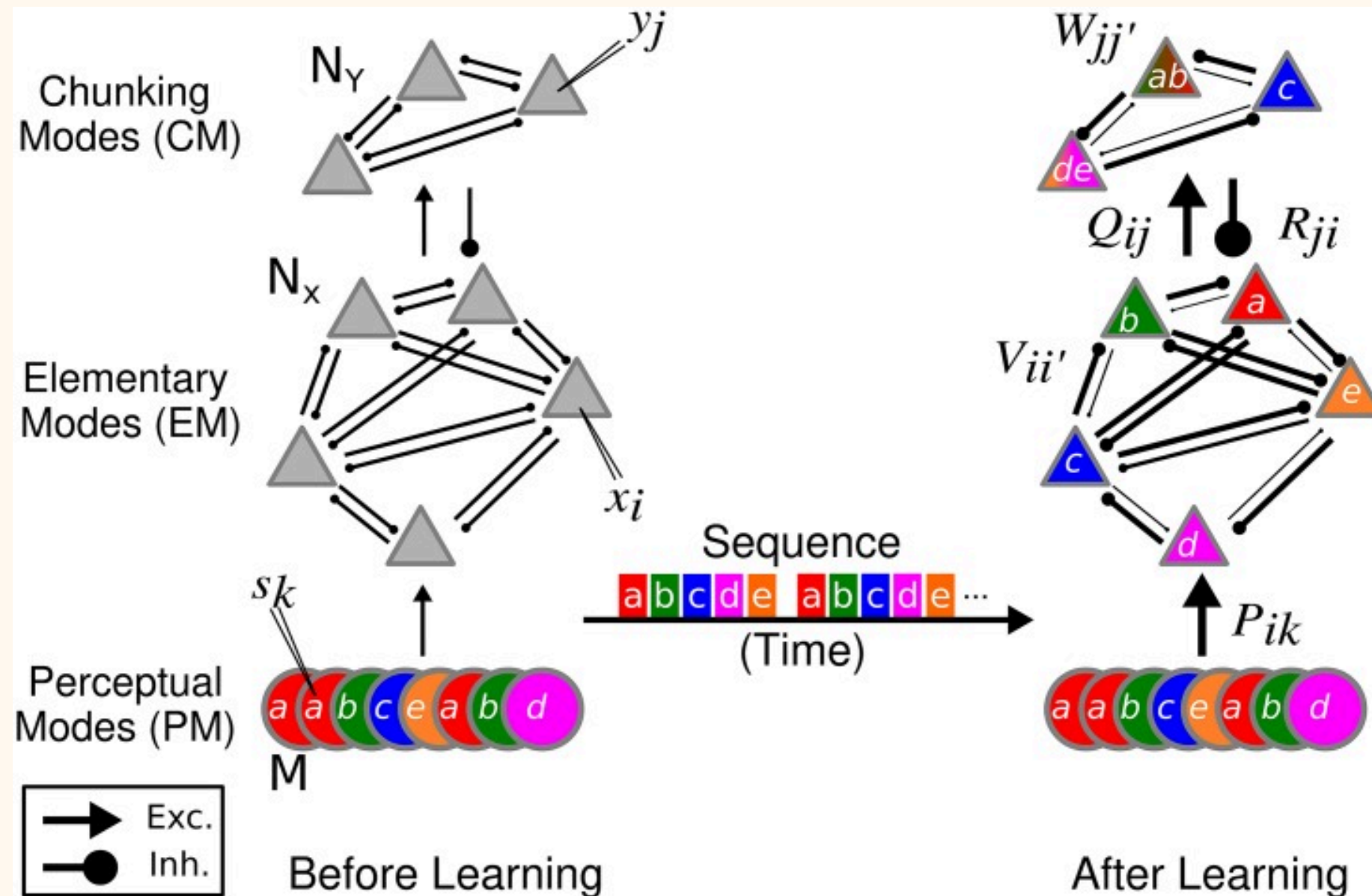


Cortical-Striatal Loop

- Enables sequence learning by encoding for action-outcome associations
- Dopaminergic neurons in this loop fire in response to prediction errors

Seger & Miller 2010

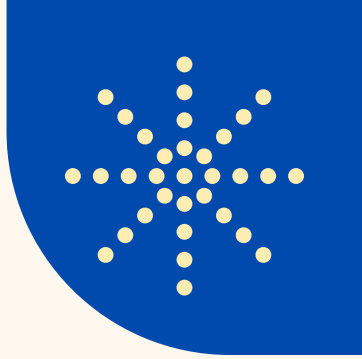
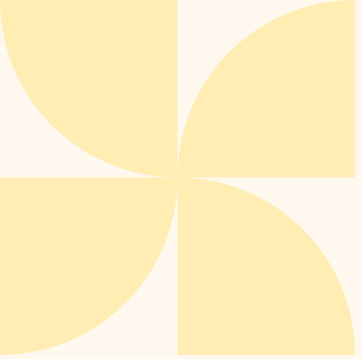
BACKGROUND



Chunked Learning

- A chunk is a collection of elements with a strong internal association but weaker associations to elements in other chunks
- Brain's sequential memory entails each chunk corresponding to a chain of metastable neural states.

Fonollosa et al. (2015)

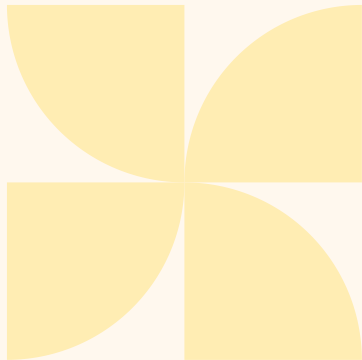
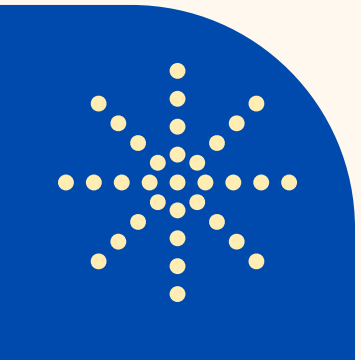


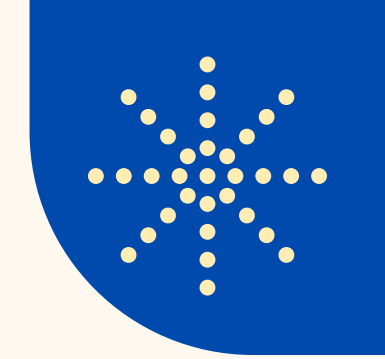
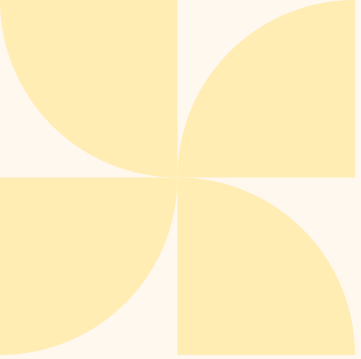
BACKGROUND

Why an RL-Model?

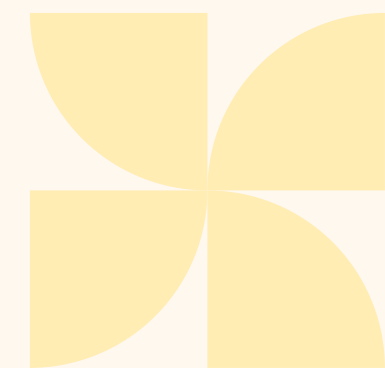
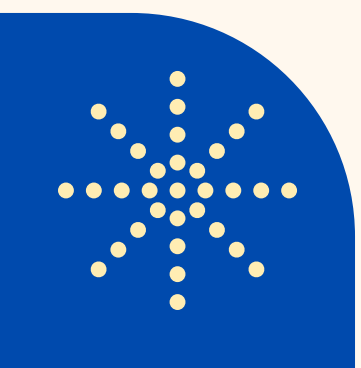
Reinforcement Learning contains several aspects that make it a favourable framework to build off of.

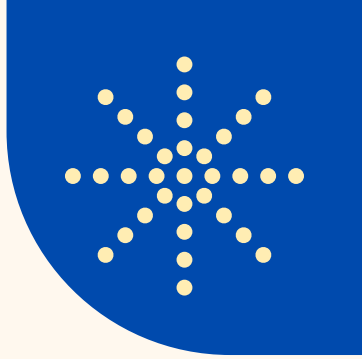
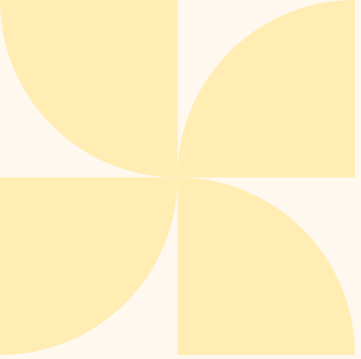
When learning a novel sequence of choreography, dancers must learn and make a sequence of decisions that will lead them to perfect the final dance. This process is not unlike an RL agent moving from state to state and making decisions based on future rewards.





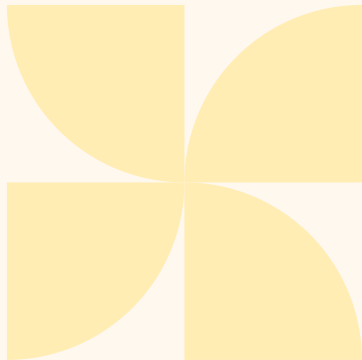
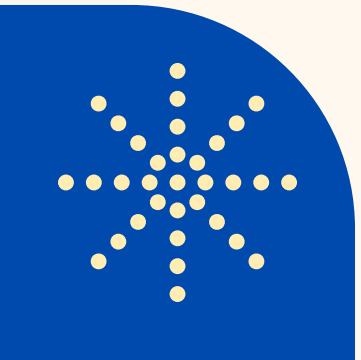
In this project, we aimed to create an **algorithm**,
using **Q-learning**, that models how **dancers best**
learn choreography!

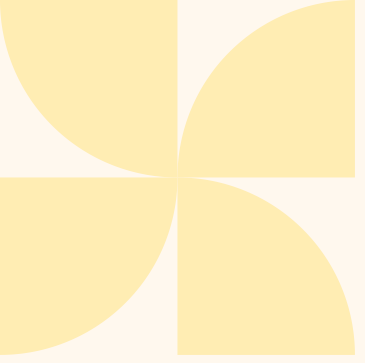




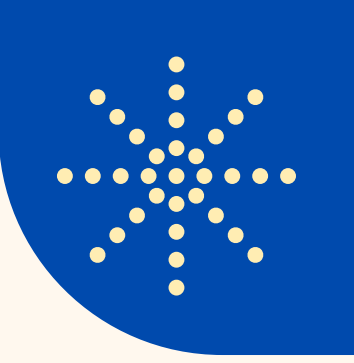
INITIAL LIMITATIONS

Learning choreography **requires the recruitment of several different brain areas** and systems. Our model simulates how dancers learn a short sequence of moves set to specific beats. It is aimed at **modeling memorization** and **sequence recognition**.

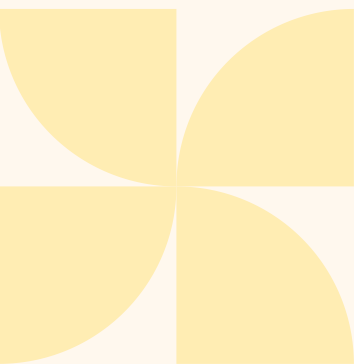
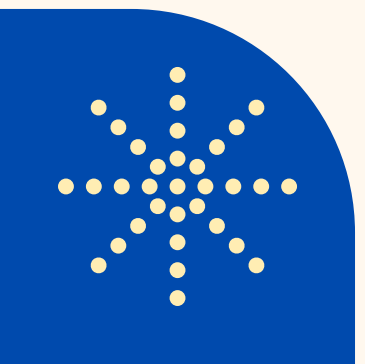




APPROACH: REWARDS



- Before we could implement our model, we had to determine the **best way to give rewards**
- When/how you reward the model matters!
- Changing the reward system helps model the **importance of a curriculum** and allows the agent to get feedback as it goes!



RANDOM

Seeing how the model performs based on luck!

```
randPrefix = zeros(1, numEpisodes);

for ep = 1:numEpisodes
    actions = randi(8, [1, 32]); % pick 32 moves completely at random

    p = 0;
    for step = 1:32
        if moves(actions(step)) == targetSeq(step)
            p = p + 1; % correct move, keep counting
        else
            break; % first mistake – prefix ends here
        end
    end
    randPrefix(ep) = p;
end
```

randomly selects a move A-H

we track when the model makes a mistake; model start over once is makes a mistake

Key Limits: Model is NOT given any rewards for correct moves and is incapable of learning/memorizing prev. actions

SPARSE REWARDS

```
Q_sparse = zeros(32, 8); % Q-table: one row per step, one column per possible move
sparsePrefix = zeros(1, numEpisodes);
```

```
for ep = 1:numEpisodes
    % epsilon decays over time – agent explores less as it gains confidence
    epsilon = max(0.01, epsilon0 * exp(-ep / 5000));

    actions = zeros(1, 32);
    for step = 1:32
        if rand < epsilon
            actions(step) = randi(8); % explore: try something random
        else
            [~, actions(step)] = max(Q_sparse(step, :)); % exploit: pick the move it thinks is best
        end
    end
end
```

**uses a Q-table to
recognize prev actions
and learn!**

```
% only learn if the entire sequence was perfect, otherwise no update at all
if all(moves(actions) == targetSeq)
    for step = 1:32
        Q_sparse(step, actions(step)) = Q_sparse(step, actions(step)) + alpha * 32;
    end
end
```

```
% evaluate greedily, what prefix would the agent get if it always picked its best move?
p = 0;
for step = 1:32
    [~, a] = max(Q_sparse(step, :));
    if moves(a) == targetSeq(step), p = p + 1; else, break; end
end
sparsePrefix(ep) = p;
end
```

**ALL 32 moves must be
correct for a reward**

CHUNKED/CURRICULUM

```
Q_chunk = zeros(32, 8);
chunkPrefix = zeros(1, numEpisodes);
currentMaxBlock = 1; % start with only the first chunk unlocked
windowSize = 50; % how many recent episodes to average when judging mastery
masteryThresh = 0.95; % agent must nail 95% of the current prefix on average to advance

for ep = 1:numEpisodes
    epsilon = max(0.01, epsilon0 * exp(-ep / 5000));

    for block = 1:currentMaxBlock % only train on unlocked chunks
        startMove = (block - 1) * blockSize + 1;
        endMove = block * blockSize;

        for step = startMove:endMove
            if rand < epsilon
                action = randi(8);
            else
                [~, action] = max(Q_chunk(step, :));
            end

            reward = double(moves(action) == targetSeq(step)); % 1 if correct, 0 if not

            % look one step ahead - if the next step has a high expected value,
            % that future promise gets folded into how we rate the current step
            nextMax = 0;
            if step < 32, nextMax = max(Q_chunk(step + 1, :)); end

            % bellman update: nudge the current estimate toward (reward + discounted future)
            Q_chunk(step, action) = Q_chunk(step, action) + ...
                alpha * (reward + gamma * nextMax - Q_chunk(step, action));

        end
    end
end
```

learns moves in chunks of 4; must avg. 95% to 'master'

if correct then reward = 1

takes future rewards into account

CHUNKED/CURRICULUM

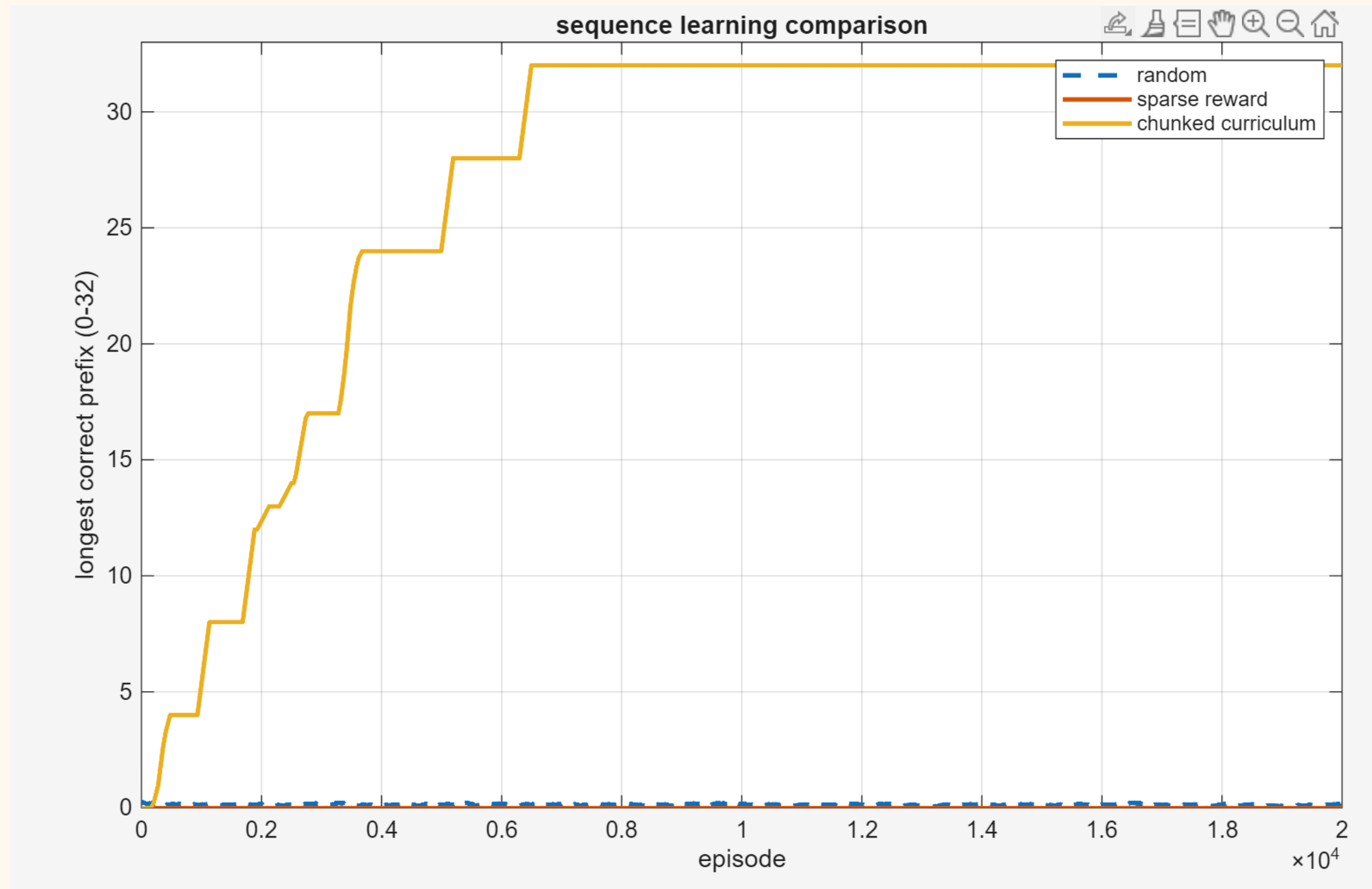
```
% measure greedy prefix, so how far does the agent get with no exploration?
p = 0;
for step = 1:32
    [~, a] = max(Q_chunk(step, :));
    if moves(a) == targetSeq(step), p = p + 1; else, break; end
end
chunkPrefix(ep) = p;

% check if the agent has earned the right to see the next chunk
if currentMaxBlock < numBlocks && ep >= windowSize
    recentPrefix = mean(chunkPrefix(ep - windowSize + 1 : ep));
    if recentPrefix >= currentMaxBlock * blockSize * masteryThresh
        currentMaxBlock = currentMaxBlock + 1;
        fprintf('episode %d: unlocked block %d\n', ep, currentMaxBlock);
    end
end
end
```

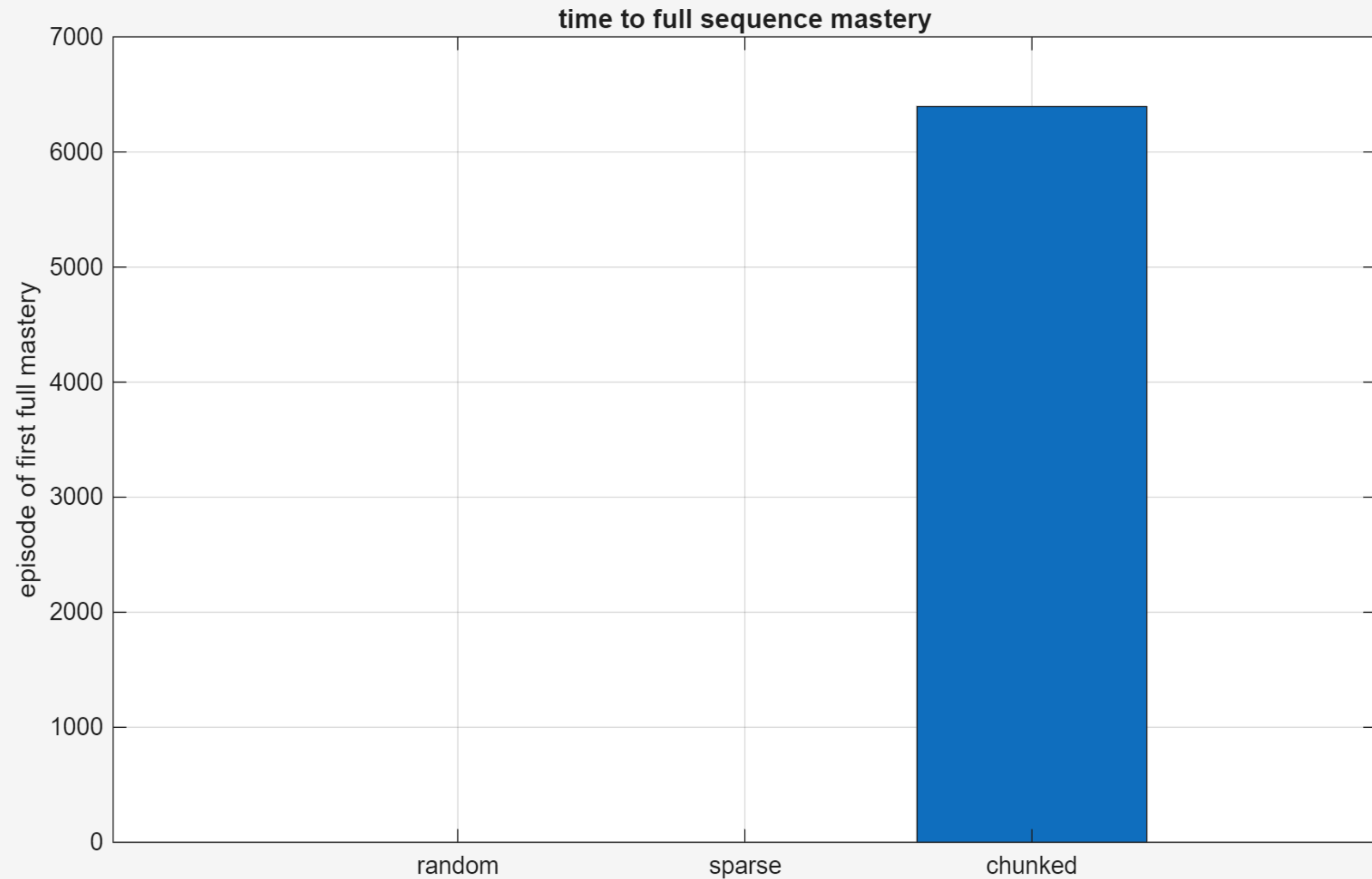
**if the agent is able to get the
current 4-move chunk 95%
correct it moves on!**

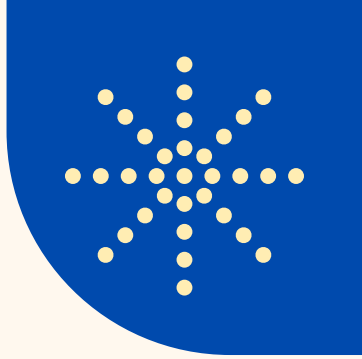
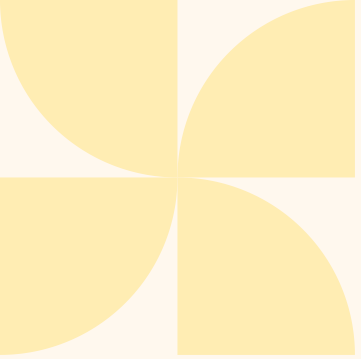


COMPARING REWARD SYSTEMS

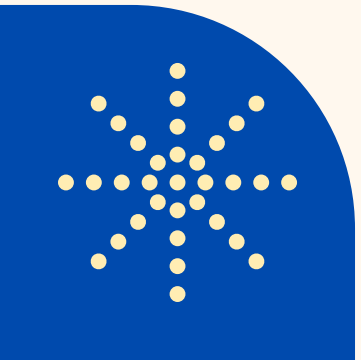
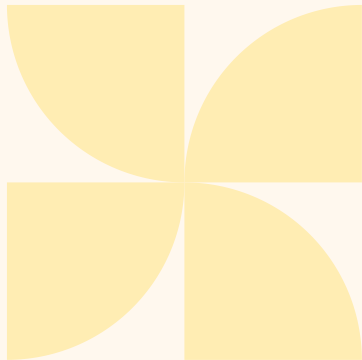


COMPARING REWARD SYSTEMS





OUR MODELS

- Yay!! Now we have a reliable method of giving rewards
 - Now we need to figure out what model works best for **learning choreography**
 - We tried **3** different models and compared their ability to learn across **differing epsilon decay rates**
- 
- 

APPROACH: MODELS

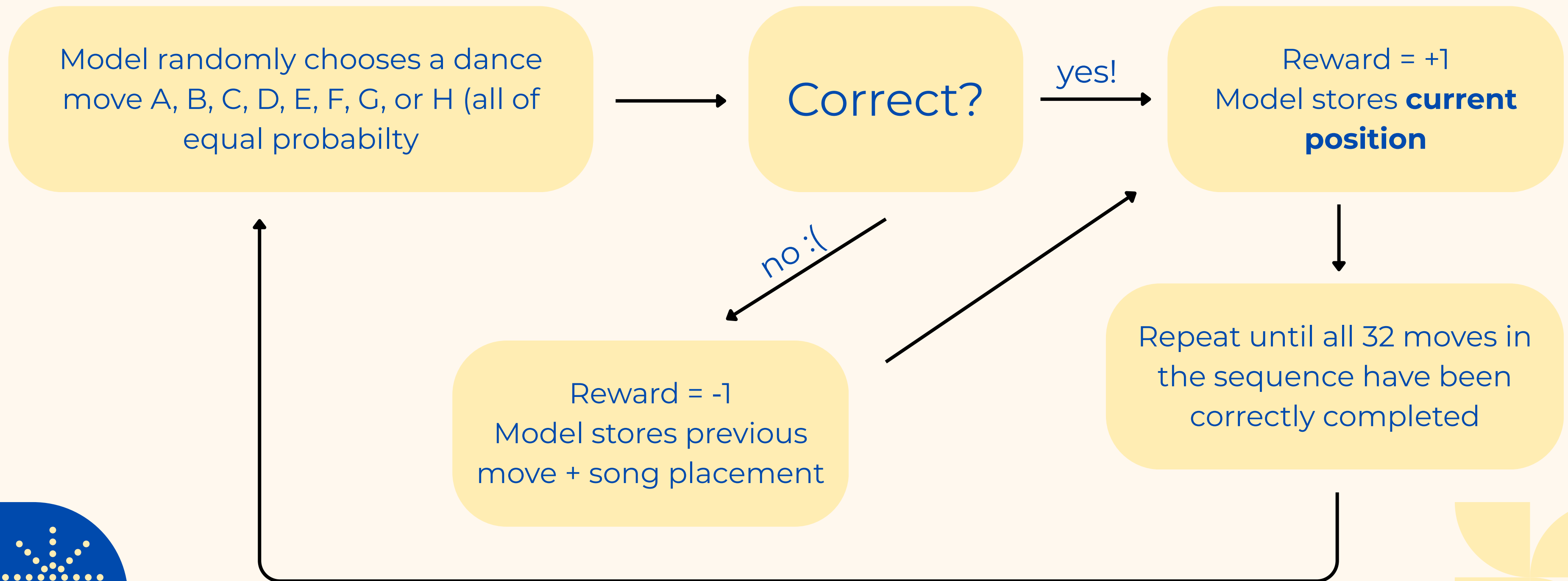
- Generate different combinations of **moves A-H**, where one **sequence is 32** moves
- 3 different models used: **Memorization, Beat + Previous Action, Beat Only**

```
episodes    = 5000;    % number of training episodes per model/decay combo
epsilon0    = 1.0;    % initial exploration rate (1 = fully random at start)
alpha       = 0.1;    % learning rate
gamma       = 0.9;    % discount factor
songid      = 1;      % which of the 100 generated sequences to train on (1-100)
targetmove  = 32;    % number of moves the agent must complete (full sequence)
beats       = 8;      % number of distinct beat positions in the rhythm cycle
```

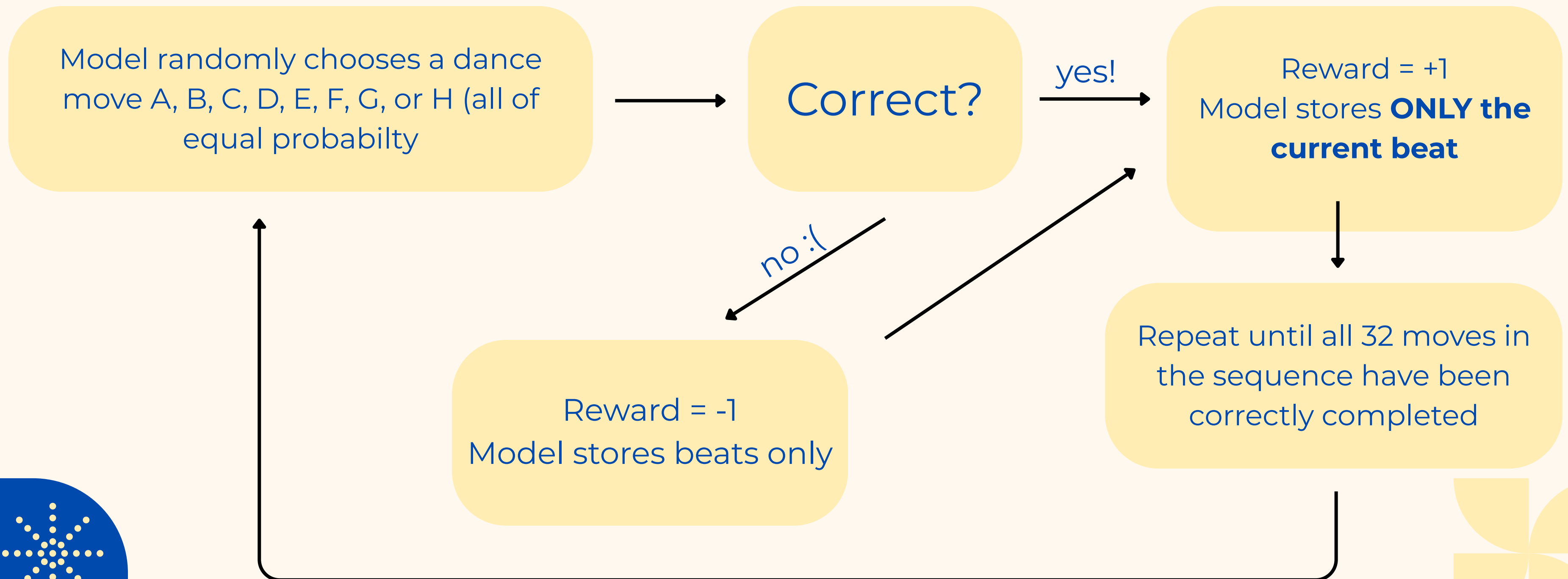
helps model know what 'music sequence' it is

similar to an '8 count' in dance: will
help the model keep its place!

MODEL 1: MEMORIZATION

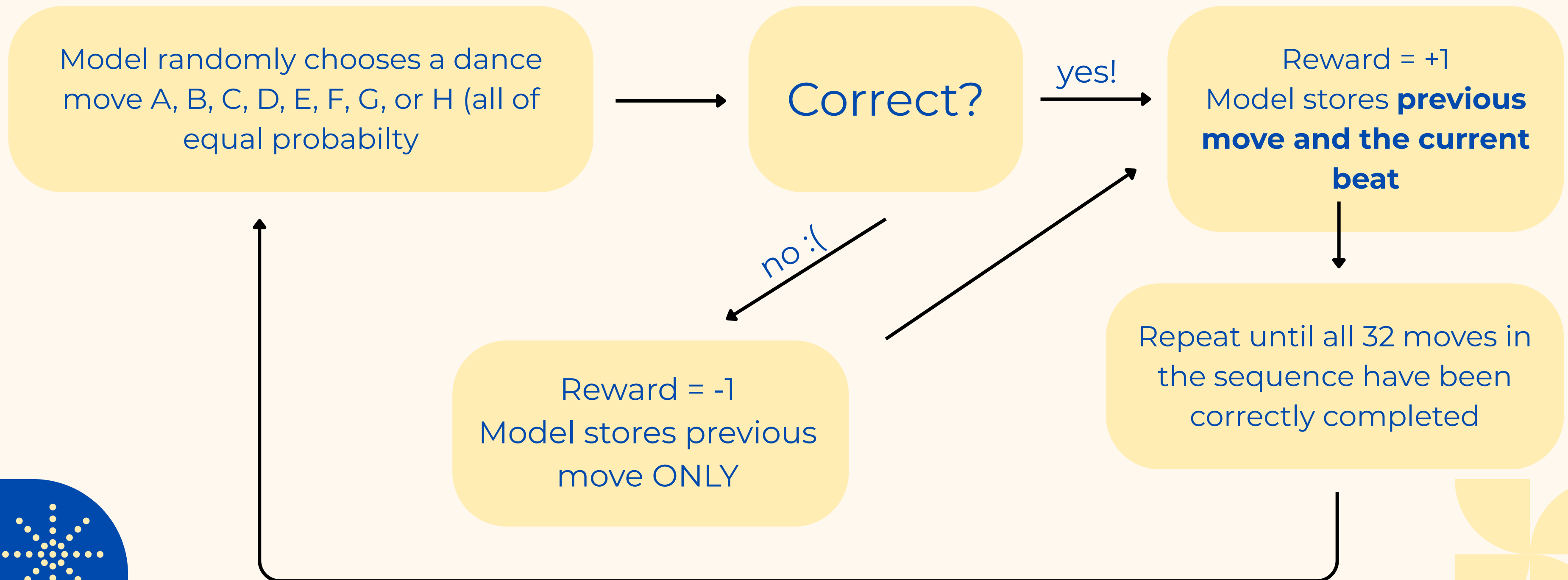


MODEL 2: BEAT

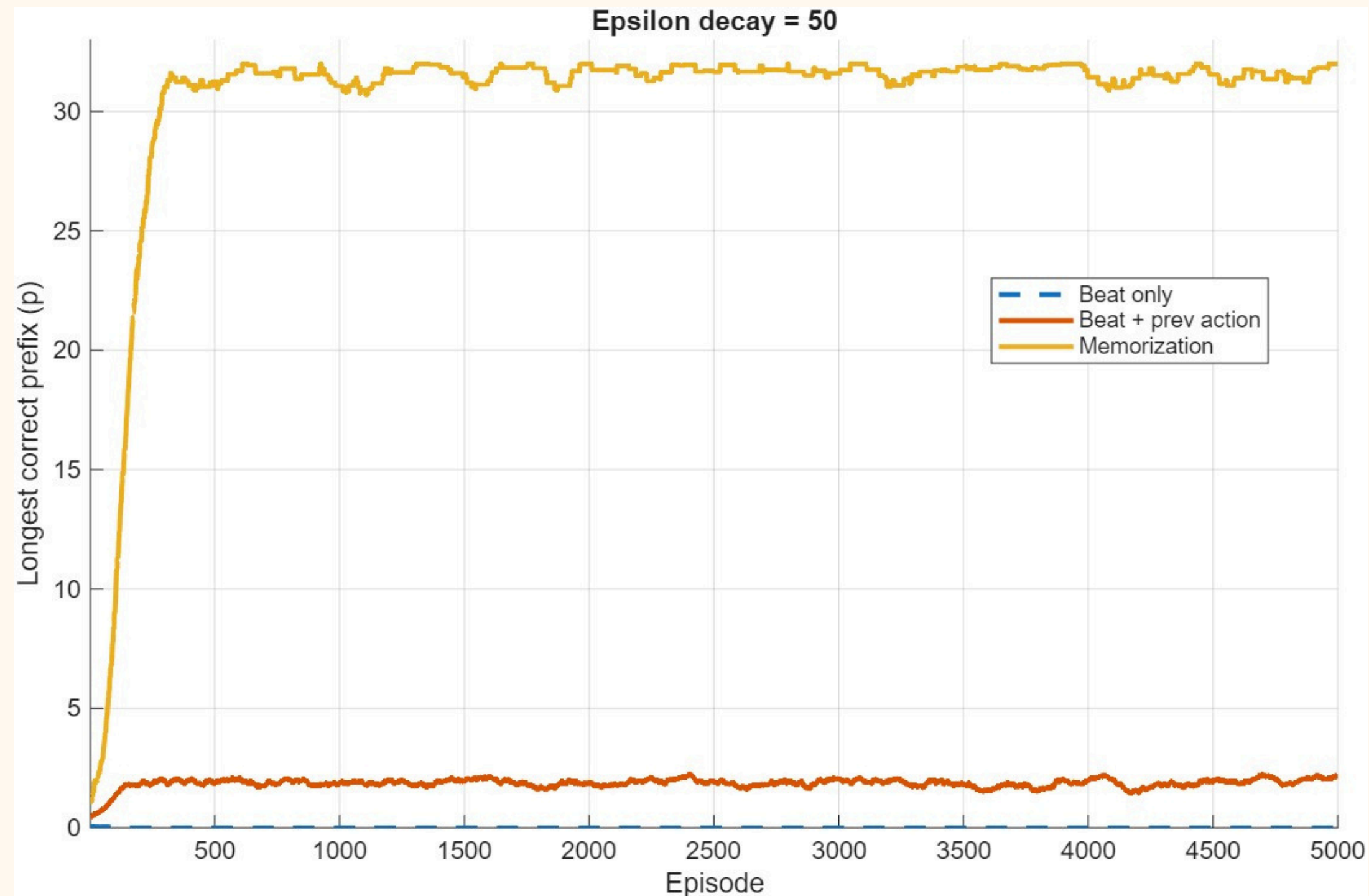


MODEL 3: BEAT + PREV.

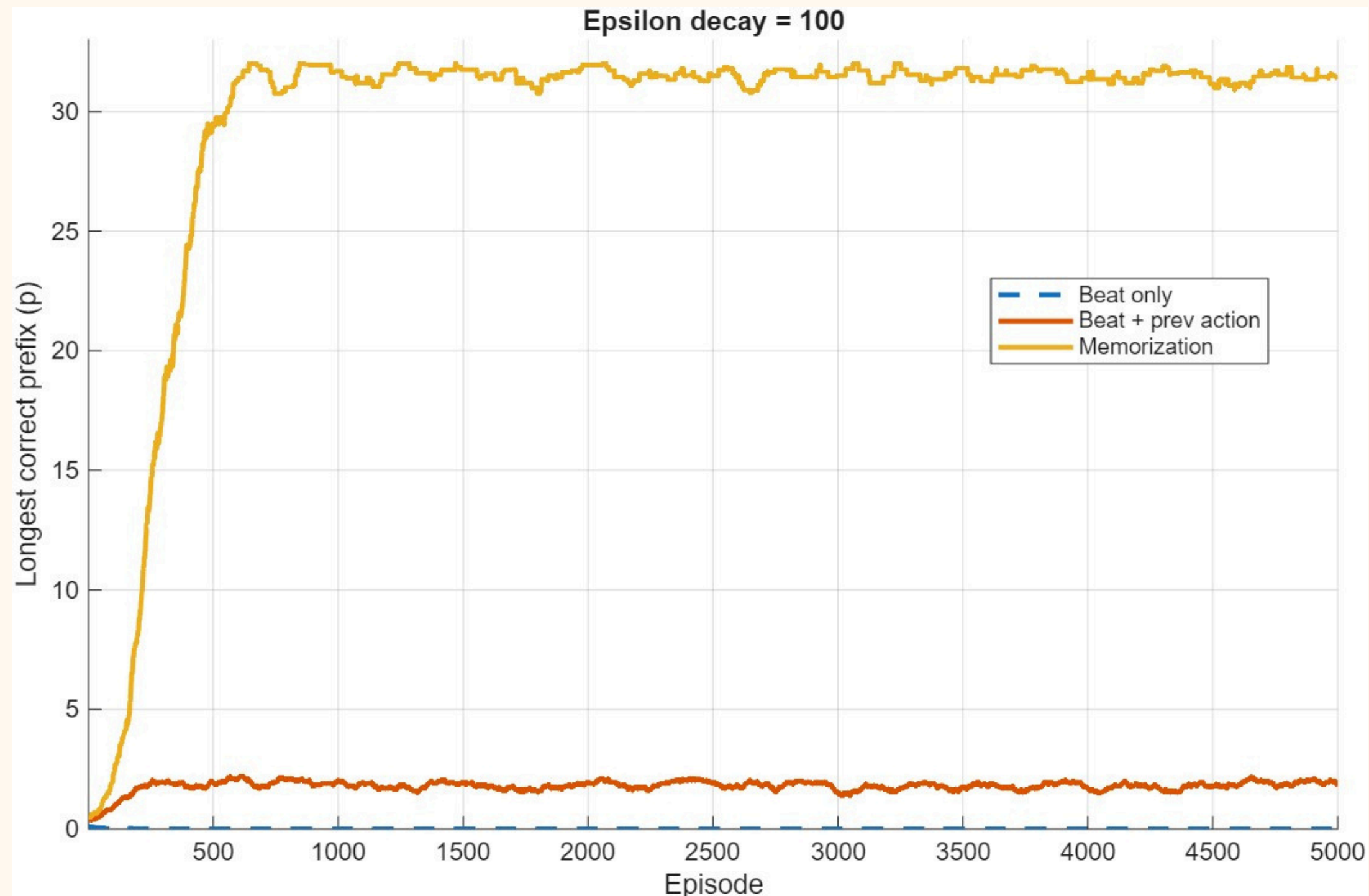
ACTION



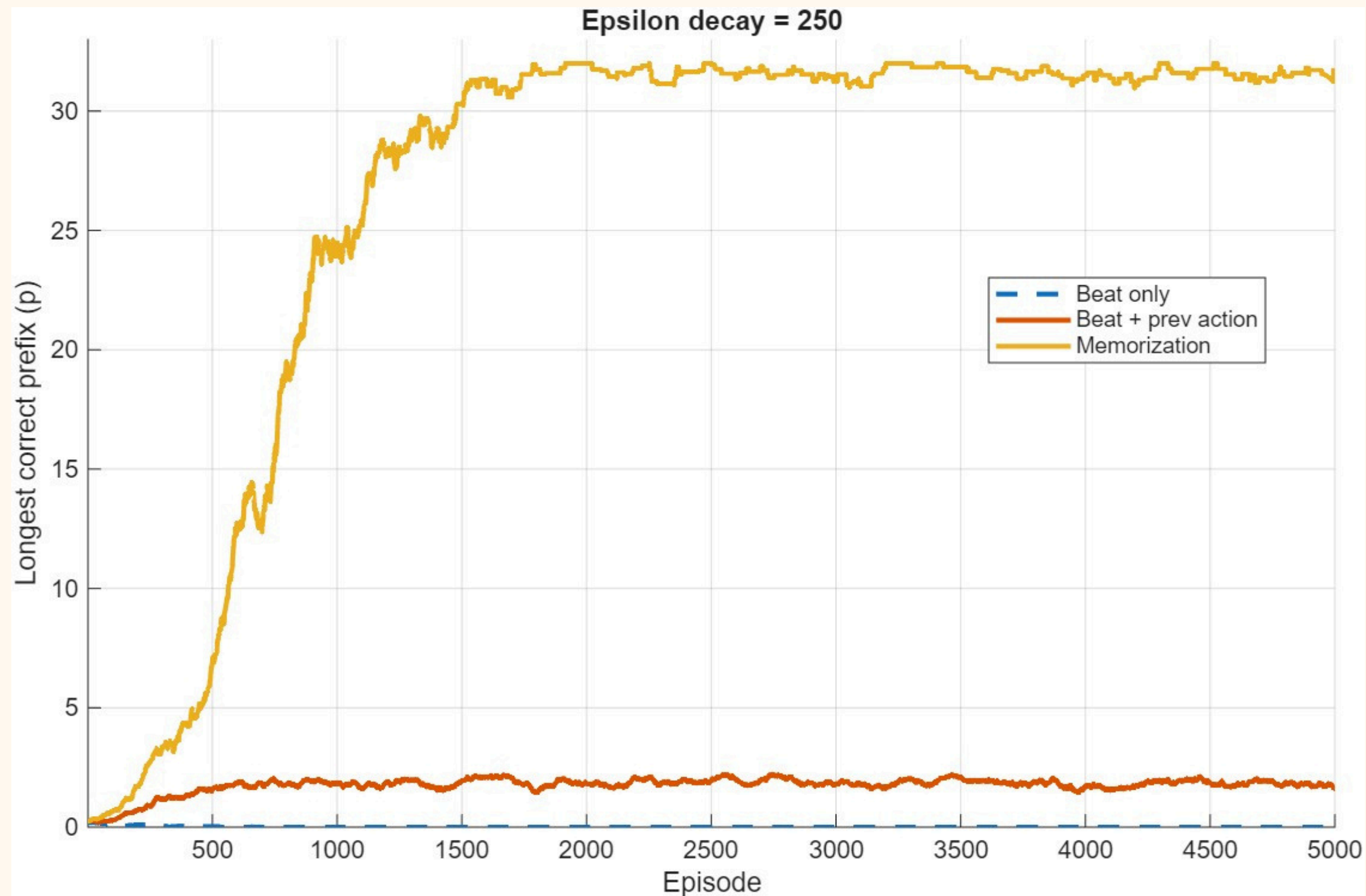
PERFORMANCE: EPSILON- DECAY 50



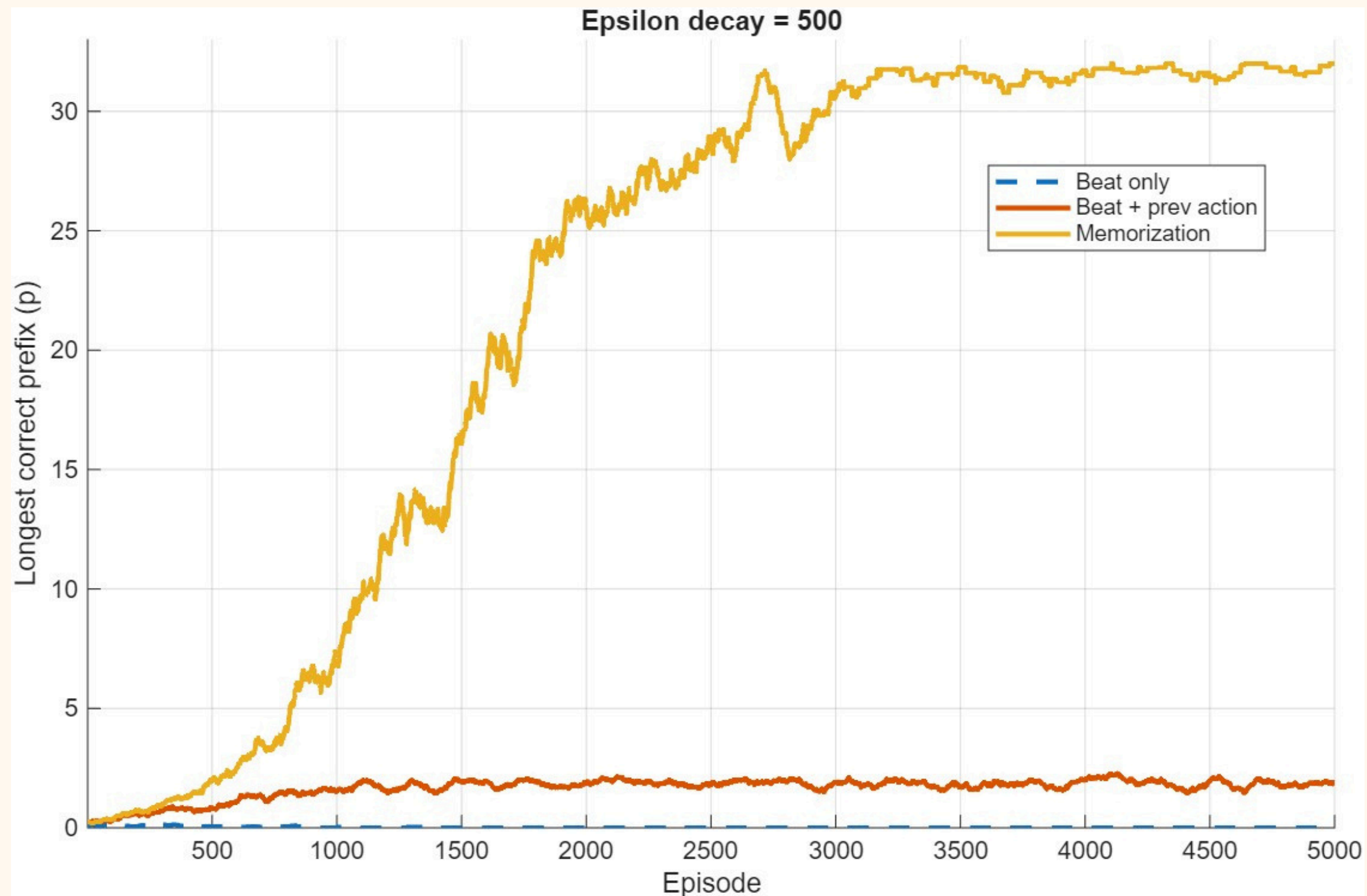
PERFORMANCE: EPSILON- DECAY 100



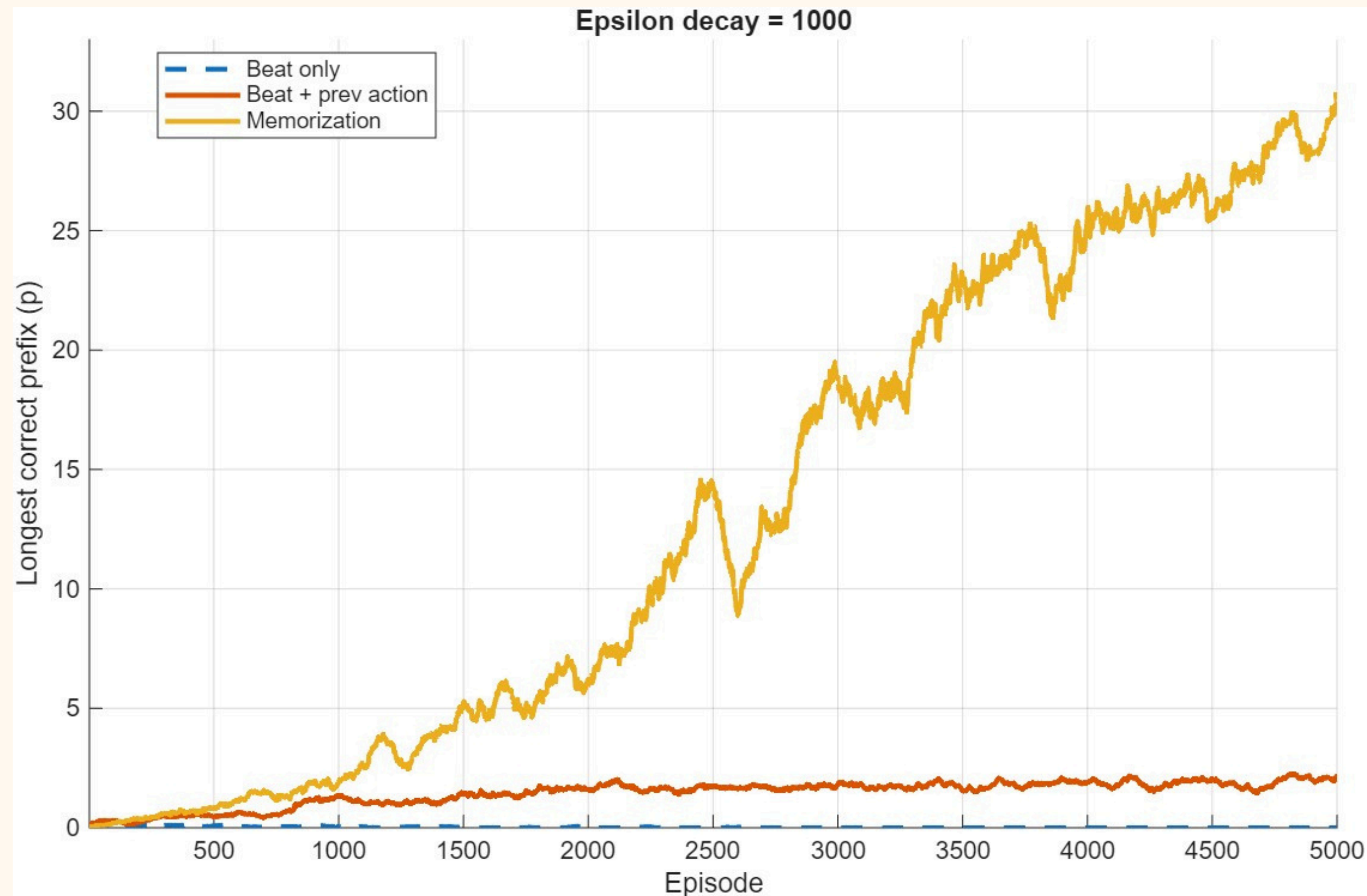
PERFORMANCE: EPSILON- DECAY 250



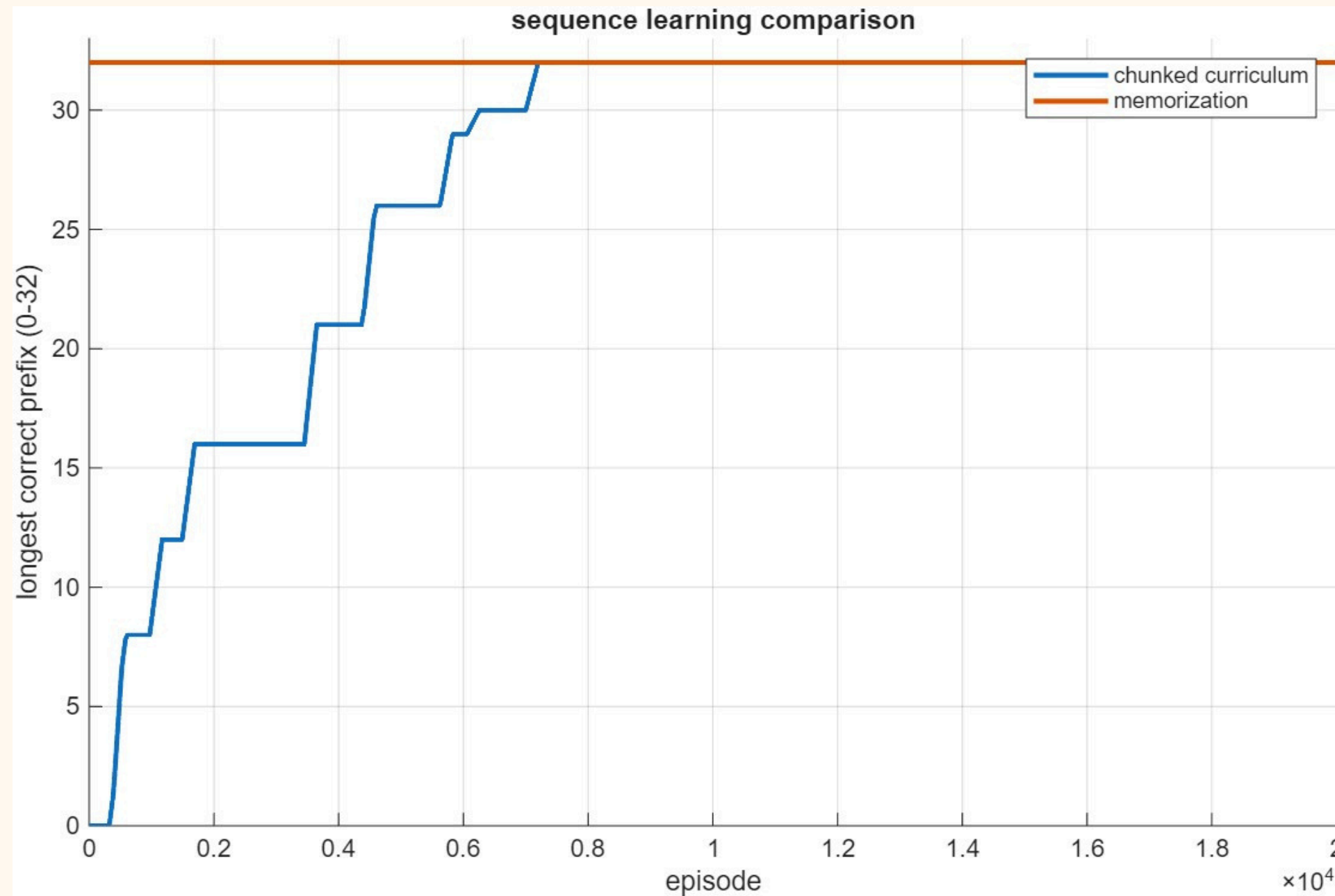
PERFORMANCE: EPSILON- DECAY 500



PERFORMANCE: EPSILON- DECAY 1000



BENEFITS & TRADEOFFS:

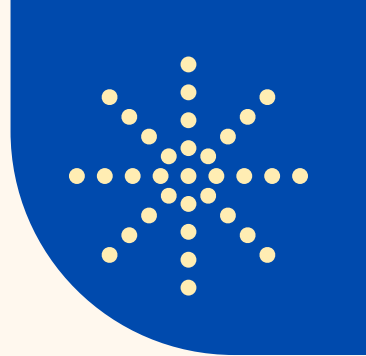
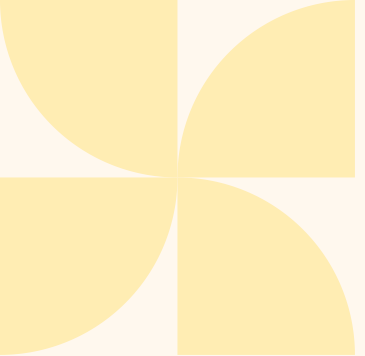


- Memorization:
 - Fast convergence
- Chunked Curriculum:
 - Reduced search space, more structured learning
 - Realistic to how humans learn

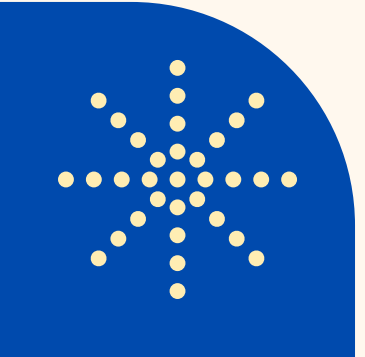
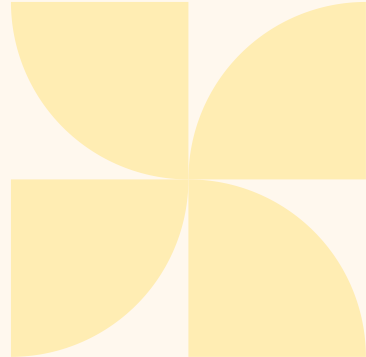
Tradeoffs:

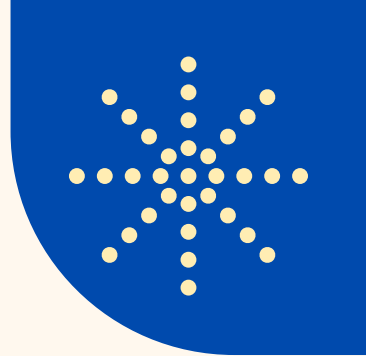
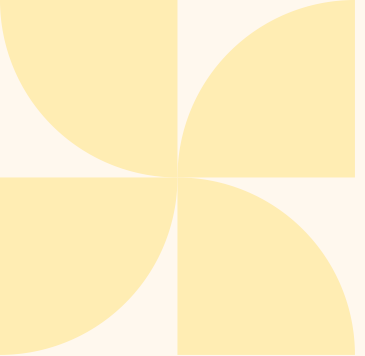
- Chunking reduces the search space, while memorization needs to search the entire space

Key insight: Both are unable to generalize when given other sequences (takes in index as a part of the state) = LESS ARTISTIC

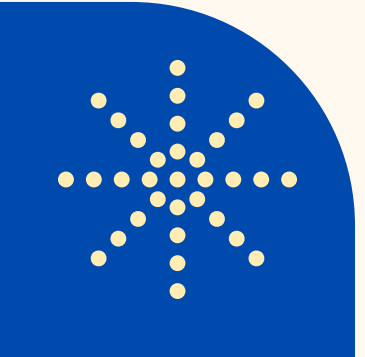
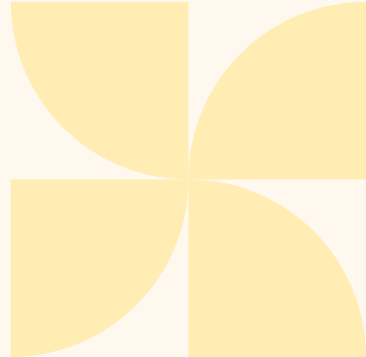


CONCLUSIONS:

- Giving **rewards throughout** the learning process is important for model learning
 - Learning in a long sequence in **chunks or pure memorization** is the best way to achieve mastery
 - Knowing where you are **in the sequence/song** is just as important as knowing the move itself
 - In the case of learning choreography, it is **better to exploit than explore!**
- 
- 



FUTURE DIRECTIONS

- Try to **represent musicality/rhythm better**, as our model only has a parameter for knowing where it is in the song, but cannot differentiate between the way things “sound.”
 - Create a model that is **more generalizable** and therefore more creative
 - Create an agent that can use models we created to learn choreography to try and **create its own choreo!**
- 
- 

FUN DEMONSTRATION!

- Now you have the chance to learn like an RL agent!
- **Person 1: Sparse Reward**
 - You will have 5 minutes to try and learn the secret combination that we came up with using the moves below!
 - The trick is... you won't get any feedback until you get ALL of them correct
- **Person 2: Chunked Curriculum**
 - You will also have 5 minutes to try and learn the secret dance, but you will learn the sequence in chunks of 2 moves at a time
 - For each move you get correct, you will receive a candy!

Dance move bank for all models!

Stomp Right

Clap

Jump

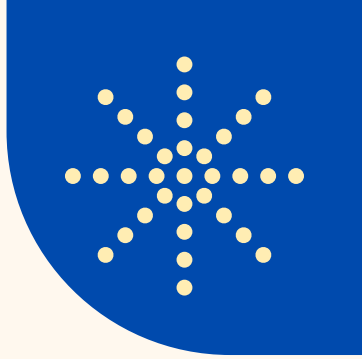
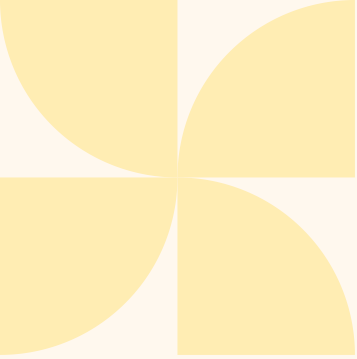
Stomp Left

Step Right

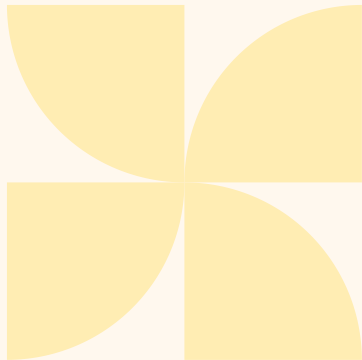
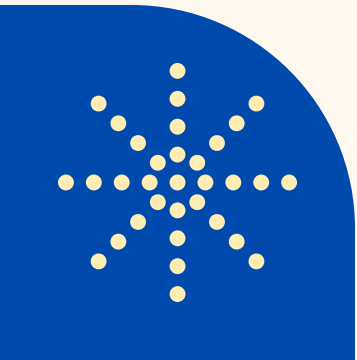
Pose

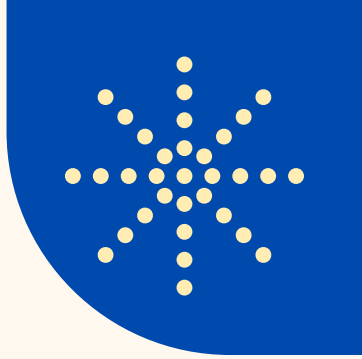
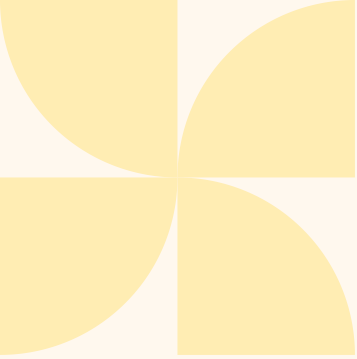
Step Left

Spin



THANK YOU!





QUESTIONS?

